

1. Теория Command Injections

1.1 Что такое Command Injection

Command Injection – уязвимость, позволяющая выполнять системные команды на сервере через несанитизированный пользовательский ввод.

OWASP Top 10: 3-е место среди самых критичных уязвимостей веб-приложений. Инъекции возникают, когда пользовательский ввод интерпретируется как часть выполняемого кода или запроса.

1.2 Типы инъекций

Тип	Описание
OS Command Injection	Ввод используется как часть OS команды
Code Injection	Ввод в функцию eval()
SQL Injection	Ввод как часть SQL запроса
XSS/HTML Injection	Ввод отображается на странице

Другие типы: LDAP Injection, NoSQL Injection, HTTP Header Injection, XPath Injection, IMAP Injection, ORM Injection. Принцип один: пользовательский ввод используется без санитизации → можно изменить логику запроса.

1.3 Причины возникновения

Использование функций для выполнения системных команд без санитизации:

Язык	Опасные функции
PHP	exec, system, shell_exec, passthru, popen
NodeJS	child_process.exec, child_process.spawn
Python	os.system, subprocess

1.4 Пример уязвимого кода

1.4.1 PHP

```
<?php
if (isset($_GET['filename'])) {
    system("touch /tmp/" . $_GET['filename'] . ".pdf");
}
?>
```

Пользовательский ввод из filename подставляется напрямую в system(). Без санитизации → можно выполнить любую команду.

1.4.2 NodeJS

```
app.get("/createfile", function(req, res){
  child_process.exec(`touch /tmp/${req.query.filename}.txt`);
})
```

1.5 Операторы инъекций

Оператор	Символ	URL-код	Поведение
Semicolon	;	%3b	Обе команды
New Line	\n	%0a	Обе команды
Background	&	%26	Обе (в фоне)
Pipe	\	%7c	Только вывод второй
AND	&&	%26%26	Вторая, если первая успешна
OR	\ \	%7c%7c	Вторая, если первая провалилась
Sub-Shell	`` `` ``	%60%60	Обе (Linux)
Sub-Shell	\$()	%24%28%29	Обе (Linux)

New Line (%0a) — часто не блокируется фильтрами, так как может быть нужен в самом payload.

1.6 Уровни защиты

Уровень	Описание
Front-end валидация	HTML5 pattern — легко обходится через Burp/curl
Back-end валидация	Проверка формата (например, IP)
Blacklist	Черный список символов/команд — обходится обфускацией
WAF	Более сложная защита — обходится продвинутыми техниками

Важно: Клиентская валидация — НЕ защита! Всегда проверяйте на бэкенде.